

Security & Observability

 systemdesignschool.io/fundamentals/cache-security-observability



The caching section has covered patterns, consistency, scaling, and failure modes. This final article addresses two often-overlooked concerns—security and observability—then provides a checklist for discussing caching in system design interviews.

Security: What Not to Cache

Caching is fundamentally about sharing data across requests. This creates risk when the cached data should not be shared.

Auth leakage. A cache key like `user:profile:123` stores one user's profile. If the application accidentally uses a key without the user identifier (e.g., `profile:current`), one user's data can be served to another user. This is a serious authorization breach.

The fix: always include the identity scope in the key. For per-user data, the key must contain the user ID. For per-tenant data, include the tenant ID. Never cache personalized data under a generic key.

Cached responses with credentials. An API response containing an access token or session cookie gets cached. The next user hitting the same cache entry receives someone else's credentials. CDNs are especially vulnerable—a `Cache-Control: public` header on an authenticated endpoint leaks tokens to all subsequent users.

The fix: never cache responses that contain tokens, cookies, or credentials. Set `Cache-Control: private, no-store` on authenticated API endpoints. Review cache headers in deployment checklists.

PII in cache. Cached user profiles containing email addresses, phone numbers, or payment data persist in memory. If the cache node's memory is accessible (debug tools, memory dumps, side-channel attacks), PII is exposed. Cached data also bypasses database-level encryption and access controls.

Mitigations:

- Cache only the fields needed for display (name, avatar), not sensitive fields (SSN, payment info)
- If sensitive fields must be cached, encrypt values at rest in the cache
- Set short TTLs on PII-containing entries to minimize exposure window
- Include cache stores in your data retention and GDPR deletion procedures

Observability: Four Key Metrics

A cache without monitoring is a black box. These four metrics reveal whether the cache is healthy, correctly sized, and protecting the database.

Hit rate. $\text{cache_hits} / (\text{cache_hits} + \text{cache_misses})$. The most important metric. Many read-heavy workloads aim for 90%+ hit rates, though acceptable targets vary by system. A drop signals that the working set exceeds cache capacity, TTLs are too short, or a failure mode (penetration, avalanche) is occurring.

Latency (p50, p95, p99). The time to complete a cache operation. On a low-latency network, Redis reads are often sub-millisecond to a few milliseconds depending on payload size and load. A spike in p99 latency indicates network congestion, oversized values, or the cache node approaching CPU saturation.

Eviction rate. How many entries Redis evicts per second due to memory pressure. Some evictions at max memory are expected, though sustained high rates may indicate undersizing. A sudden spike means the working set just grew (new feature launch, traffic increase) and the cache needs more memory.

Memory usage. Current memory consumption vs maximum configured memory. When memory reaches `maxmemory`, Redis evicts keys if an eviction policy is configured (otherwise writes are rejected with OOM errors). Track the ratio over time to forecast when capacity needs to increase.

Example starting thresholds (tune based on your baseline and SLOs):

- Hit rate drops below 85% for 5 minutes
- p99 latency exceeds 5ms
- Eviction rate increases 10x from baseline
- Memory usage exceeds 90% of configured maximum

This sets up a caching interview checklist—a structured framework with detailed examples for each dimension.